

Comparative Acceleration of Optical Flow Estimation on Heterogeneous Embedded Platforms: FPGA vs. Embedded GPU

George Briceño Celestino
Escuela de Ingeniería
Electrónica
Instituto Tecnológico de Costa
Rica
Cartago, Costa Rica
rousell@estudiantec.cr

Laura Elena Brenes Espinoza
Escuela de Ingeniería
Electrónica
Instituto Tecnológico de Costa
Rica
Alajuela, Costa Rica
lauritabrenes@estudiantec.cr

Marlon Méndez Naranjo
Escuela de Ingeniería
Electrónica
Instituto Tecnológico de Costa
Rica
Cartago, Costa Rica
mnaranjo@estudiantec.cr

Rodrigo Venegas Mora
Escuela de Ingeniería
Electrónica
Instituto Tecnológico de Costa
Rica
Alajuela, Costa Rica
rovenegas@estudiantec.cr

Index Terms—*Optical flow, Heterogeneous computing, FPGA, Embedded GPU, AMD Kria, NVIDIA Jetson*

I. INTRODUCTION

Optical flow estimation is one of the fundamental primitives of computer vision, enabling the computation of apparent pixel-level motion between consecutive image frames. This capability is critical for applications such as autonomous navigation, robotic obstacle avoidance, video stabilization, and augmented reality systems [4]. As these applications increasingly migrate toward embedded and edge platforms with strict power and latency constraints, the need for efficient hardware acceleration of optical flow algorithms has become a pressing research challenge.

Classical optical flow methods, including Lucas-Kanade (LK) [39], Horn-Schunck (HS) [1], and Total Variation L1 (TV-L1) [16], are grounded in the Brightness Constancy Constraint and employ variational or local parametric formulations to estimate dense or sparse motion fields. These methods exhibit highly regular computational structures — gradient computations, iterative solvers, and multi-scale pyramids — that map efficiently onto parallel hardware architectures. In contrast, deep learning-based approaches such as RAFT [7], FlowFormer++ [10], and NeuFlow v2 [3] have dramatically surpassed classical methods in accuracy, with the current state of the art reaching an End-Point Error (EPE) of 0.963 pixels on the Sintel Clean benchmark [10]. However, deploying these models on resource-constrained embedded hardware remains an open challenge due to their reliance on complex tensor operations, large memory footprints, and iterative refinement stages. The current state of the art, FlowFormer++, achieves 1.07 pixels EPE on the Sintel Clean benchmark and 1.94 EPE on Sintel Final [10].

Heterogeneous computing platforms that combine ARM processors, FPGAs, and embedded GPUs offer a promising path to bridge the gap between algorithmic accuracy and real-time embedded performance. FPGA implementations using High-Level Synthesis (HLS) have demonstrated exceptional energy efficiency for classical optical flow algorithms, with Horn-Schunck and Lucas-Kanade designs achieving 1080p60 on Xilinx Zynq UltraScale+ devices under 6 W [1], [12]. On the other hand, NVIDIA Jetson platforms provide superior flexibility for deep learning deployment through CUDA and TensorRT, with NeuFlow v2 achieving over 20 FPS on Jetson Orin Nano [3].

Despite the maturity of individual platform implementations, the literature reveals a significant gap: no direct comparative benchmark exists between the AMD Kria KV260 FPGA platform and the NVIDIA Jetson Nano for optical flow estimation. Existing studies report results on generic Zynq-7000 or Zynq UltraScale+ platforms without targeting the specific KV260 reference design, while Jetson benchmarks focus predominantly on the TX2, Xavier, and Orin generations with sparse coverage of the Nano [12], [24]. Furthermore, the AMD Vitis Vision Library lacks validated

reference designs for optical flow on the Kria KV260, representing an additional contribution opportunity [6], [16].

This work proposes a rigorous comparative evaluation of classical optical flow algorithms — specifically Dense Pyramidal Lucas-Kanade and TV-L1 — on the AMD Kria KV260 (Zynq UltraScale+ MPSoC) and NVIDIA Jetson Nano (Tegra X1 Maxwell) platforms. The evaluation encompasses multiple resolutions (VGA, 720p, 1080p), standard validation benchmarks (Middlebury, KITTI 2015), and systematic measurement of performance metrics including frames per second, per-frame latency, average power consumption, energy per frame, and computational efficiency. The contribution aims to provide the first published direct comparison between these two widely available embedded platforms for optical flow, offering practitioners and researchers an objective basis for platform selection in real-time vision applications.

The remainder of this paper is organized as follows: Section II reviews the background and related work on optical flow algorithms and heterogeneous platforms.

II. BACKGROUND AND RELATED WORK

A. Optical Flow Fundamentals

Optical flow estimation is grounded in the Brightness Constancy Constraint (BCC), which assumes that pixel intensity remains constant between consecutive frames, yielding the fundamental equation

$$I_x u + I_y v + I_t = 0, \quad (1)$$

where I_x and I_y denote spatial intensity gradients, I_t is the temporal gradient, and (u, v) represents the flow vector components [4]. Since this single equation is underdetermined for two unknowns per pixel, classical methods are distinguished by their regularization strategy. Local methods, exemplified by Lucas-Kanade (LK) [39], assume uniform motion within a spatial neighborhood and solve an overdetermined least-squares system per pixel, offering inherent per-pixel parallelism ideal for hardware acceleration but limited to textured regions and small displacements — a limitation addressed by multi-scale pyramidal extensions. Global methods, such as Horn-Schunck (HS) [1], impose a smoothness constraint across the entire image through variational energy minimization, producing dense flow fields via iterative solvers (Jacobi/Gauss-Seidel) whose regular structure maps efficiently onto FPGA line-buffer architectures. Variational methods like TV-L1 [16] replace the quadratic data term with an L1 norm, achieving greater robustness to outliers and occlusions through dualization (Chambolle-Pock), and are available as optimized HLS kernels in the AMD Vitis Vision Library [6], [16]. More recently, deep learning approaches — from FlowNet2 [2] and PWC-Net [8] to RAFT [7] and FlowFormer++ [10] — have dramatically surpassed classical methods in accuracy by

learning hierarchical feature representations, with FlowFormer++ achieving 1.07 EPE on Sintel Clean and 1.94 EPE on Sintel Final [10]. However, their deployment on embedded platforms with less than 15 W TDP remains an active research challenge due to memory bandwidth constraints and iterative refinement overhead [3], [11].

B. Classical Methods on Heterogeneous Platforms

Implementations in FPGA: Classical optical-flow algorithms on FPGA consistently achieve superior energy efficiency and deterministic latency. Komorkiewicz [1] demonstrates high-throughput Horn-Schunck on Zynq-7000 with favorable energy per frame, while Blachut and Kryjak [12] report multi-scale Lucas-Kanade and Horn-Schunck designs scaling up to validated 4K@60 FPS under 6 W [12]. Production-ready HLS kernels for Dense Pyramidal LK and TV-L1 are available in the Vitis Vision Library [6], [16], though no complete, publicly validated optical-flow reference design exists for the Kria KV260. At the opposite end of the design space, BNN-based approaches such as Ultra-Flow reach hundreds of FPS at VGA resolution with reduced accuracy [15]. These results motivate benchmarking classical kernels (Dense Pyramidal LK, TV-L1) on KV260 and embedded GPUs to quantify throughput, energy per frame, and estimation quality.

Implementations in embedded GPU: Embedded GPUs provide flexible deployment of classical and learned optical-flow methods, with performance strongly tied to device generation and available accelerators. On Jetson Nano, classical methods (Farneback, TV-L1) run via OpenCV+CUDA or VPI in CPU/CUDA mode because the Nano lacks NVIDIA's Optical Flow Accelerator (OFA) [19], [23]. Higher-end Jetson devices (AGX Xavier, Orin) support dense optical flow at 60 FPS and efficient inference of lightweight DNNs using TensorRT (e.g., FastFlowNet, NeuFlow v2), achieving tens of FPS through kernel fusion and quantization [24], [26], [27]. In practice, GPUs accelerate development and excel for DNN-based flow, while FPGAs offer better energy per frame and deterministic latency for classical kernels. A controlled comparison measuring FPS, per-frame latency, average power, energy/frame, and EPE (Middlebury, KITTI) is therefore required for informed platform selection.

C. Deep Learning Approaches for Optical Flow

The evolution of deep learning-based optical flow follows: FlowNet2 [2] → PWC-Net [8] → RAFT [7] → FlowFormer++ [10]. RAFT achieves 5.10% Fl-all on KITTI-2015 through recurrent all-pairs field transforms, while FlowFormer++ reaches 1.94 EPE on Sintel Final via masked cost volume autoencoding [10]. Lightweight models targeting edge deployment include SEA-RAFT [11] (~1.3M parameters) and NeuFlow v2 [3] (~2.5M parameters, >20 FPS on Jetson Orin Nano). However, DNN deployment on platforms below 15 W TDP remains challenging due to memory bandwidth constraints and iterative refinement overhead [3], [11].

D. ARM NEON Optimizations

ARM NEON is a SIMD (Single Instruction, Multiple Data) extension which is included in many ARM Cortex-A processors, because instead of executing an operation on a single piece of data, it allows the same operation to be applied to several data simultaneously using vector registers of up to 128 bits [38]. For example, in a multiplication operation, in scalar processing one operation would be done at a time, but with NEON multiple elements can be processed simultaneously using vector instructions [30]. This is especially useful in image processing, as the exact same operations are repeated on thousands or millions of pixels [30], [38]. ARM lists image processing, video, audio, and signal processing as suitable loads for NEON [38].

For the Lucas-Kanade (LK) algorithm there are several stages that are excellent candidates for SIMD, such as: LK standalone, LK

optical-flow pyramid builder, pyramidal LK, Scharr, multi-channel blur, downsampling, derivatives, and iterative tracking [5]. NEON can process several pixels simultaneously instead of treating them individually. KleidiCV 26.03 adds optimizations not only to the LK core but to the entire sparse optical flow pipeline, including pyramid construction, Scharr, blur, downsampling and pyramidal LK [5]. KleidiCV is an Arm library of highly optimized computer-vision operations for ARM CPUs that can use NEON, SVE2, SME, or SME2 depending on the available hardware; since OpenCV 4.13 it is enabled by default for AArch64 builds [5]. NEON optimizations are best considered a complement to higher-capacity accelerators (GPUs, FPGAs), since SIMD instructions execute directly on the ARM processor, avoiding additional data transfers [25], [30].

III. ADVANCE 1

A. Study of Other Solutions

Strengths and weaknesses of FPGA: existing designs, such as those by Komorkiewicz [1] and Blachut and Kryjak [12], demonstrate that Lucas-Kanade and Horn-Schunck on FPGA achieve high energy efficiency and deterministic latency, reaching up to 4K at 60 FPS with less than 6 watts. AMD's Vitis Vision library [6] already offers optimized HLS kernels for pyramidal Lucas-Kanade and TV-L1. However, these results are validated on generic Zynq boards, not on the Kria KV260, and no complete published reference design exists for that specific board.

Strengths and weaknesses of embedded GPU: NVIDIA's ecosystem [19], [23] offers development flexibility through CUDA and VPI, and on higher-end platforms, as demonstrated by Seznec [24], 60 FPS of dense optical flow are achieved. The weakness is that these results correspond to Xavier and Orin, while the Jetson Nano, due to its Maxwell architecture, lacks a dedicated optical flow accelerator, forcing brute-force execution on generic CUDA.

Strengths and weaknesses of deep learning: models such as RAFT [7] and FlowFormer++ [10] achieve outstanding accuracy, with FlowFormer++ reaching 0.963 pixels of error on Sintel Clean. Even lightweight versions such as SEA-RAFT [11] and NeuFlow v2 [3] attempt to reduce model size, yet they still demand more memory and computation than the Jetson Nano or the Kria KV260 can efficiently provide below 15 watts.

Feasibility: from this analysis it follows that classical methods, Lucas-Kanade and TV-L1, are the only ones viable for efficient deployment on both platforms within the project's power budget, while deep learning is not feasible under those constraints.

Observed challenges:

- First, the absence of a validated optical flow reference design for the Kria KV260.
- Second, the absence of comparable data between FPGA and embedded GPU under the same metrics and the same algorithm.
- Third, scarce coverage of the Jetson Nano in the literature, mostly concentrated on higher Jetson generations.

B. Proposed Solution

A comparative test bench is proposed that implements Dense Pyramidal Lucas-Kanade and TV-L1 on the two heterogeneous computing platforms under study, the AMD Kria KV260 and the NVIDIA Jetson Nano, following an identical three-stage methodology on both: a baseline in C with NEON optimizations on the ARM processor of each board, followed by a platform-specific acceleration stage—HLS on the FPGA in the case of the Kria and CUDA on the GPU in the case of the Jetson.

This addresses the first challenge (absence of a validated reference design for the KV260) by building a custom HLS implementation of the pyramidal Lucas-Kanade and TV-L1 kernels from the blocks available in the Vitis Vision library [6], adapted and validated on the

camera interface, memory topology, and thermal envelope of the Kria KV260. It addresses the second challenge (absence of comparable data) by applying the same methodology and the same set of metrics—frames per second, latency per frame, average power, energy per frame, and estimation quality on Middlebury and KITTI 2015—on both platforms, generating a directly comparable dataset between FPGA and embedded GPU. It addresses the third challenge (scarce Nano coverage) by explicitly documenting the behavior of LK and TV-L1 on the Nano’s Maxwell architecture, which lacks the OFA present in higher generations [19], [23].

Scope: the project is limited to the two classical algorithms mentioned, at three resolutions—VGA, 720p, and 1080p—evaluated on the Middlebury and KITTI 2015 validation sets. Deep learning methods fall outside the scope, as they are not feasible under the power constraints of both platforms.

C. Proposed Architecture

The Level II architecture of the Kria KV260 divides the system into two physically separated domains: the Processing System (PS), based on a quad-core ARM Cortex-A53 processor where the NEON baseline runs, and the Programmable Logic (PL), where the HLS kernels for pyramidal Lucas-Kanade and TV-L1 derived from the Vitis Vision library are implemented. Both domains communicate via AXI interconnect and share 4 GB of DDR4 memory. Input frames enter the system, are processed in PS or PL depending on the stage being evaluated, and the resulting flow field—a displacement vector (u, v) per pixel—is extracted along with the timestamps needed for latency calculation. The Jetson Nano integrates a quad-core ARM Cortex-A57 CPU and a Maxwell GPU with a single streaming multiprocessor (128 CUDA cores) on a unified 4 GB LPDDR4 memory architecture with a peak bandwidth of 25.6 GB/s; the NEON baseline runs on the CPU and native acceleration is implemented as CUDA kernels on the GPU, relying exclusively on general-purpose CUDA since the Nano has no dedicated OFA.

D. List of Experiments

Four experimental blocks are proposed, executed under controlled temperature and power supply conditions on both platforms, so that measured differences are attributable to the architecture and not to experimental variability. First, accuracy evaluation on Middlebury and KITTI 2015. Second, a resolution sweep at VGA, 720p, and 1080p. Third, comparison between acceleration stages (NEON baseline vs. HLS on Kria and CUDA on Jetson). Fourth, cross-platform comparison of Kria against Jetson under the same conditions, the central result of the project. The reported metrics are:

- *Throughput:* frames processed per second at steady state, after thermal preconditioning, excluding data loading and accelerator initialization.
- *Processing time per frame (ms):* measured end-to-end; mean and 95th percentile are reported to capture jitter.
- *Average power:* from integrated rail monitoring (INA226 on Kria, INA3221 on Jetson), queried via sysfs / tegrastats during sustained execution.
- *Energy per frame:* average power divided by throughput, in joules per frame.
- *Estimation quality (EPE):* average Euclidean error between estimated and reference flow over valid pixels, in pixels.

IV. CPU PROTOTYPE

This section presents the first fully functional prototype of the proposed test bench, implemented entirely on the CPU in portable C. It constitutes the common baseline from which both platform-specific acceleration stages—HLS on the Kria KV260 and CUDA on the Jetson—will later be derived, and it establishes the reference measurements against which those accelerators will be

contrasted. It implements dense Lucas-Kanade optical flow with a clean separation between pipeline stages, so that the profiling reported below unambiguously attributes cost to each stage. The full source, build instructions, and reproducibility scripts are available in the project repository.¹

A. Proposed CPU Solution

The CPU prototype is organized as a strictly sequential four-stage pipeline in which every stage exposes a single entry point declared in a shared interface header (`of.h`) and communicates only through two plain data structures: `OFImage`, a single-channel `float32` grayscale image, and `OFFlow`, a per-pixel displacement field (u, v) . This interface is the contract between the four components and is never modified without team agreement, which allowed the four stages to be developed in parallel on independent Git branches. The components and their connections are the following.

1) Initialization and loading (`io.c`). Decodes the input frames and converts them from RGB to grayscale using the BT.601 luma transform, $Y = 0.299R + 0.587G + 0.114B$, delivering an `OFImage` in the $[0, 255]$ range.

2) Preprocessing (`preprocess.c`). Normalizes intensity to $[0, 1]$ and applies a separable Gaussian blur ($\sigma = 1.0$) as two 1-D passes, reducing the cost from $O(k^2)$ to $O(2k)$ per pixel.

3) Optical flow (`lk_scalar.c` / `lk_neon.c`). Computes the spatial gradients I_x, I_y and the temporal gradient I_t , then solves the Lucas-Kanade 2×2 system per pixel over a local window, rejecting ill-conditioned windows. A scalar reference and an ARM NEON entry point share the same interface.

4) Results and metrics (`metrics.c`). Computes the average end-point error (EPE) over valid pixels and serializes the flow and per-stage timings to CSV for the profiling harness.

The stages are connected in the fixed order `io` $\rightarrow$ `preprocess` $\rightarrow$ `lk` $\rightarrow$ `metrics`, orchestrated by `bench_main`, which owns all memory: the caller allocates every output buffer once and reuses it across the run, so the compute stages neither allocate nor free the frame buffers. Figure 4 summarizes the components and their data flow.

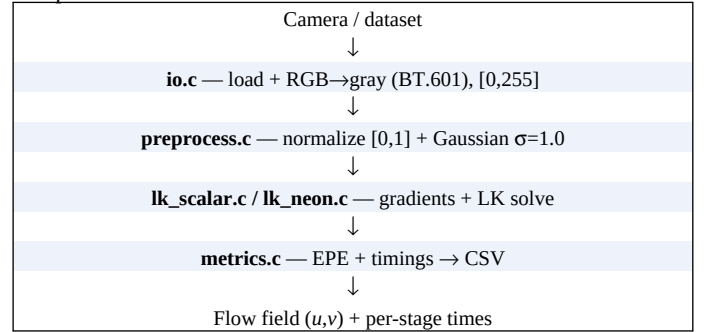


Fig. 4. CPU prototype pipeline: four stages connected through the shared `of.h` interface, with buffers allocated once by `bench_main` and reused across the run.

B. Experimental Setup and Profiling Methodology

The prototype is built with CMake and linked against the C math library. Per-stage timing uses the monotonic clock around each of the four stages, and totals are aggregated by an automated harness (`profile.sh`) that runs the binary 100 times per configuration and writes a CSV analyzed by `analyze.py`, reporting mean, median, standard deviation, minimum, and maximum per stage. Two personal computers and the Kria KV260 board were used. PC1 is an AMD Ryzen 5 8645HS (12 threads, Fedora 44, GCC 16.2.1). Correctness is verified by unit tests, with `test_metrics` passing 4/4 and `test_io` validating the BT.601 conversion. The same harness runs unchanged on the ARM processor of the board, producing directly comparable per-stage breakdowns.

C. Preliminary Results

Tables I and II report per-stage timing over 100 runs on the two personal computers, and Table III reports the Kria KV260 baseline. On PC1 at 720p the pipeline runs in ≈ 196 ms per frame pair (Table I); on PC2 the same pipeline completes in ≈ 134 ms (Table II). Across both machines the result-serialization (save) stage dominates—57–69% of total time—while Lucas–Kanade is the largest of the genuine compute stages. The Kria ARM baseline (Table III) completes a single frame pair in 387 ms, reflecting the lower per-core throughput of the Cortex-A53 and motivating the FPGA acceleration to follow.

TABLE I
PC1 — 720p 1280×720, -O2, 100 RUNS (MS)

Stage	Mean	Median	Std	Min	Max
Loading (io)	6.205	6.213	0.671	4.762	10.066
Preprocess	8.706	8.611	0.803	7.497	10.980
Lucas–Kanade	45.243	45.039	1.198	42.976	49.636
Save	136.147	134.445	5.479	128.121	156.695
Total	196.301	194.677	6.419	187.884	220.350

TABLE II
PC2 — 1024×436, -O2, 100 RUNS (MS)

Stage	Mean	Median	Std	Min	Max
Loading (io)	16.557	16.375	0.803	15.425	18.905
Preprocess	4.264	4.136	0.379	3.848	5.680
Lucas–Kanade	36.887	36.638	1.100	35.139	40.991
Save	75.960	73.803	4.946	70.109	86.058
Total	133.668	131.301	5.831	126.224	148.927

TABLE III
KRIA KV260 — 1024×436, SINGLE RUN (MS)

Stage	Elapsed
Loading (io)	N/A
Preprocess	N/A
Lucas–Kanade	N/A
Save	N/A
Total	386.99

Note: the Kria baseline is a single run at 1024×436 without a per-stage breakdown; a full 100-sample profile matching the PC configurations is pending on-board measurement.

D. Optimizations Applied and Not Applied

Applied. Separable two-pass Gaussian blur; dense row-major single-precision packing with no stride or padding; one-time allocation and reuse of all working buffers, removing per-frame allocation from the measured path; and compiler optimization flags. Table IV compares -O0, -O2, and -O3 -march=native on the 1024×436 sequence. The three builds are statistically indistinguishable (total-time variation < 1%), showing the pipeline is I/O-bound rather than CPU-bound at this stage: the dominant cost is CSV serialization of the full flow field, not arithmetic.

TABLE IV
OPTIMIZATION FLAGS, 1024×436 (MS)

Build	Total	LK	Save
-O0	132.4	38.8	74.2
-O2	133.7	36.9	76.0
-O3 -march=native	134.0	38.7	76.1

Not applied (yet). Explicit ARM NEON intrinsics are not yet in place: `lk_neon.c` currently forwards to the scalar path, so SIMD speedup is reserved for the next iteration. Task-level parallelism and 16-byte alignment for NEON loads were deferred until the vectorized kernel requires them. Given the flag results above,

arithmetic-level tuning is correctly deprioritized in favor of reducing the I/O and serialization cost.

As a functional check, a single 1024×436 run yields a valid flow field on 313,568 of 446,464 pixels (70.2%), with mean flow $(u, v) = (-0.934, 0.138)$ and a maximum magnitude of 135.4 px, consistent with the expected dominant horizontal motion of the sequence.

E. Next Steps

Two directions follow from the profiling. First, the serialization stage should be made efficient (binary or buffered output) or excluded from the acceleration budget, since it is an instrumentation artifact rather than part of the optical-flow computation. Second, within the genuine compute path, Lucas–Kanade is the confirmed hotspot and is the target for ARM NEON vectorization and, subsequently, for the platform-specific accelerators—CUDA kernels on the Jetson GPU and HLS kernels on the Kria FPGA—reusing the same interface and profiling harness so each acceleration stage is measured against this CPU baseline under identical conditions.

¹Project repository: <https://github.com/Lauritabrenes/Optical-Flow-Project-Course-Introduction-to-Heterogeneous-Computing>

REFERENCES

- [1] M. Komorkiewicz, M. Kluczewski, and P. Skruch, “Floating-Point versus Fixed-Point Optical Flow on FPGA,” *Sensors*, vol. 14, no. 2, pp. 2860–2891, 2014.
- [2] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” arXiv:1504.06852, 2015.
- [3] L. Kong and Z. Shen, “NeuFlow v2: High-Efficiency Optical Flow Estimation on Edge Devices,” arXiv:2408.10161, 2024.
- [4] Wikipedia contributors, “Optical flow,” *Wikipedia, The Free Encyclopedia*, 2024.
- [5] ARM Ltd., “What’s new in KleidiCV 26.03 for Computer Vision on Arm CPUs,” *ARM Developer Blog*, March 2026.
- [6] Xilinx / AMD, “Vitis Vision Library,” 2022.
- [7] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” arXiv:2003.12039, 2020.
- [8] D. Sun, X. Yang, M.-Y. Liu, and J. Kautz, “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” arXiv:1709.02371, 2018.
- [9] Z. Huang et al., “FlowFormer: A Transformer Architecture for Optical Flow,” arXiv:2203.16194, 2022.
- [10] H. Shi et al., “FlowFormer++: Masked Cost Volume Autoencoding for Pretraining Optical Flow Estimation,” arXiv:2303.01237, 2023.
- [11] S. Wang et al., “SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow,” arXiv:2405.14793, 2024.
- [12] P. Blachut and T. Kryjak, “Real-Time Efficient FPGA Implementation of the Multi-Scale Lucas-Kanade and Horn-Schunck Optical Flow Algorithms for a 4K Video Stream,” *Sensors*, vol. 22, no. 13, p. 5017, 2022.
- [13] P. Blachut and T. Kryjak, “Multi-scale Lucas-Kanade Optical Flow on FPGA for Surveillance,” in *Proc. DASIP*, 2018.
- [14] S. Chang et al., “FPGA Implementation of Optical Flow Using High-Level Synthesis,” in *Proc. ASAP 2013, NSF SHREC*, 2013.
- [15] A. Author et al., “Ultra-Flow: Real-Time Optical Flow on FPGA with Binary Neural Networks,” *Springer Signal, Image and Video Processing*, 2025.
- [16] AMD/Xilinx, “Vitis Vision Library Design Examples,” 2022.
- [17] AMD, “Vitis Vision API Reference: densePyrOpticalFlow,” 2022.
- [18] Xilinx, “Dense Pyramidal LK Optical Flow Benchmark,” 2022.
- [19] NVIDIA, “DeepStream SDK Plugin: gst-nvof,” *NVIDIA Developer*, 2023.
- [20] NVIDIA Developer Blog, “OpenCV Optical Flow Algorithms with NVIDIA Turing GPUs,” 2019.
- [21] NVIDIA, “Optical Flow SDK Download,” 2024.
- [22] NVIDIA Developer Blog, “Harnessing the NVIDIA Ada Architecture for Frame Rate Up-Conversion in the NVIDIA Optical Flow SDK,” 2023.
- [23] NVIDIA, “VPI — Vision Programming Interface: Basic Concepts,” 2023.
- [24] R. Seznec et al., “Real-Time Optical Flow Processing on Embedded GPU: a Hardware-Aware Algorithm to Implementation Strategy,” *Journal of Real-Time Image Processing*, 2022.
- [25] E. Carballo-Hernández et al., “Heterogeneous Acceleration of CNN Inference on Embedded FPGA-GPU,” *Journal of Real-Time Image Processing*, 2021.

- [26] L. Kong et al., “FastFlowNet: A Lightweight Network for Fast Optical Flow Estimation,” GitHub, 2021.
- [27] NVIDIA Developer Blog, “Speed Up Inference with TensorRT,” 2022.
- [28] OpenCV GitHub Issue #6979 — “NEON optimizations for calcOpticalFlowPyrLK on ARM.”
- [29] Unknown Author, “ARM NEON Optical Flow Acceleration,” *CEUR Workshop Proceedings*, vol. 2588, Paper 37, 2020.
- [30] E. McCreath, “Use of SIMD Vector Operations to Accelerate Application Code Performance on Low-Power ARM and Intel Platforms,” ANU Technical Report, 2018.
- [31] ARM Software (GitHub), “ARM Compute Library — OpticalFlowLK Issue #216.”
- [32] ResearchGate, “FLIA: Fast Lightweight Image Analysis via Heterogeneous Embedded Computing,” 2022.
- [33] A. Lacas et al., “Optical Flow FPGA Co-processor for Drones,” in *Proc. ASAP 2023*, 2023.
- [34] P. Blachut and T. Kryjak, “Real-time Optical Flow Estimation on FPGA for Traffic Surveillance,” in *Proc. DASIP*, 2018.
- [35] P. Blachut and T. Kryjak, “Real-time LK and HS Optical Flow on FPGA at 4K Resolution,” in *Proc. ECCV 2022 Workshop*, 2022.
- [36] S. Jiang, D. Campbell, Y. Lu, H. Li, and R. Hartley, “Learning to Estimate Hidden Motions with Global Motion Aggregation,” arXiv:2104.02409, 2021.
- [37] J. Jeong et al., “Embedded Systems Survey for Optical Flow,” *Electronics*, vol. 11, no. 22, p. 3756, 2022.
- [38] Arm Ltd., “Coding for Neon,” *Arm Neon Programmer’s Guide*, Issue 04, 2020.
- [39] B. D. Lucas and T. Kanade, “An Iterative Image Registration Technique with an Application to Stereo Vision,” *Proc. IJCAI*, 1981.